

## Exercise 6: Connecting to the .NET API

**Context:** Mock data has brought you this far. Now you will connect your Angular client frontend directly to your running .NET Web API microservice.

### Asynchronous Architecture: Observables vs. Signals

Up to this point, signals and computed dependencies have behaved as synchronous primitives within your templates. Conversely, the framework `HttpClient.get()` method returns an **Observable**—a lazy execution stream that emits its asynchronous value over time before completing. Someone must explicitly subscribe to open the stream and receive data payloads.

To bridge this boundary cleanly and avoid memory leaks from manual setups, we leverage `rxResource`. This wrapper encapsulates the asynchronous RxJS streaming logic inside the service layer while exposing simple signals (`.value()`, `.isLoading()`, and `.error()`) straight to the rendering templates.

[!NOTE]

#### Step 1: Verify Your API Layer Status

Spin up your .NET backend context from a separate terminal instance to confirm that it exposes valid JSON catalogs rather than raw root-level arrays:

```
cd path/to/your/tms-api
dotnet run
```

Verify your live integration payloads using cURL or Postman:

- **V1/M6 Contracts (GET /api/courses):** The catalog items are wrapped inside an envelope object where the actual rows occupy the `items` array property.
- **V2 Contracts (GET /api/v2/courses):** The model rows are wrapped within the `data` array property, while metadata maps underneath the `meta` nested block.

*Integration Mapping Strategy:* If you consume a V2 endpoint schema, remember to update your frontend RxJS `.pipe(map(...))` operator chains from `p.items` to `p.data` to match the data model format exactly.

[!NOTE]

#### Step 2: Create the Course Service

Generate a single shared service layer using the Angular CLI tool schema:

```
ng generate service services/course --type=service
```

Open `src/app/services/course.service.ts` and replace the code skeleton with this dependency configuration mapping toward your .NET endpoint:

```
import { Service, inject } from "@angular/core";
import { HttpClient } from "@angular/common/http";
import { map } from "rxjs/operators";
import { Course, PagedResponse } from "../models/course.model";

@Service()
public class CourseService {
  private http = inject(HttpClient);
  private baseUrl = "https://localhost:5001/api/courses";

  getAll(page = 1, pageSize = 50) {
    return this.http
      .get<PagedResponse<Course>>(this.baseUrl, {
        params: { page: page.toString(), pageSize:
        pageSize.toString() },
      })
      .pipe(map((p) => p.items)); // Update target properties to
      p.data if integrating against V2 routes
  }

  getById(id: string) {
    return this.http.get<CourseDetail>(`\${this.baseUrl}/\${id}`);
  }
}
```

*Design Note: The modern `@Service()` decorator establishes this class as an app-wide singleton provider instance, matching the functionality of `AddSingleton<T>()` inside the .NET dependency injection engine.*

[!NOTE]

### Step 3: Consume the Service with rxResource

Open your existing `student-dashboard.component.ts`. Clean out your hardcoded `availableCourses` signal arrays along with the supporting mock objects. Merge the modern asynchronous tracking resource configuration directly into your controller layout:

```
import { Component, signal, computed, inject } from
"@angular/core";
import { rxResource } from "@angular/core/rxjs-interop";
import { CourseCardComponent } from "../../ui/course-card/course-
card.component";
import { CourseService } from "../../services/course.service";

@Component({
  selector: "app-student-dashboard",
  standalone: true,
```

```

imports: [CourseCardComponent],
templateUrl: "./student-dashboard.component.html",
styleUrl: "./student-dashboard.component.scss",
})
export class StudentDashboardComponent {
  private api = inject(CourseService);

  studentName = signal("Liya Kebede");
  earnedCredits = signal(45);
  selectedCourse = signal<Course | null>(null);

  graduationStatus = computed(() =>
    this.earnedCredits() >= 120 ? "Eligible for Graduation" : "In
Progress",
  );

  // rxResource safely manages under-the-hood subscriptions and
  handles cleanups automatically upon destruction
  coursesResource = rxResource({
    stream: () => this.api.getAll(),
  });

  handleEnroll(course: Course) {
    this.selectedCourse.set(course);
    console.log("Enrollment requested for:", course.title);
  }
}

```

[NOTE]

#### Step 4: Update the Template View Architecture

Open `src/app/features/student-dashboard/student-dashboard.component.html`.  
Re-wire your markup elements to evaluate the state of the managed signal streams:

```

<h2>Course Catalog</h2>

@if (coursesResource.isLoading()) {
  <div class="spinner">Fetching courses from the server...</div>
} @else if (coursesResource.error()) {
  <div class="error">
    Could not load courses. Make sure your .NET API is running.
  </div>
} @else {
  <div class="grid">
    @for (course of coursesResource.value()!; track course.id) {
      <tms-course-card [course]="course"
(enrollClicked)="handleEnroll($event)" />
    } @empty {
      <p>No courses are available this term.</p>
    }
  </div>
}

```

```
</div>
}
```

*Note: The non-null assertion operator (!) inside `coursesResource.value()!` tells the compiler the value is safe to evaluate here. This is guaranteed since the `@else` execution branch runs only after both loading and error flags evaluate to false.*

[!NOTE]

### Step 5: Configure Backend Cross-Origin Resource Sharing (CORS)

Because the client browser blocks cross-origin traffic between separate local host ports (4200 to 5001), you must register a security exception inside your .NET `Program.cs` before your HTTP requests can succeed:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAngular", policy =>
        policy.WithOrigins("http://localhost:4200")
            .AllowAnyHeader()
            .AllowAnyMethod());
});

// Enable right before mapping endpoint or controller behaviors
app.UseCors("AllowAngular");
```

[!NOTE]

### Step 6: Live Browser Verification Loop

Open `http://localhost:4200/dashboard` in your browser. Open Developer Tools (F12) and navigate straight to the **Network** telemetry dashboard tab to trace outbound requests.

*Expected Verification State:* You should see a successful **GET** request routed directly to the endpoint URL declared in your `CourseService` (e.g., `https://localhost:5001/api/courses?page=1&pageSize=50`). The server must respond with a **200 OK** status and return a wrapped envelope instead of a root-level array. Your UI components parse the incoming data rows dynamically to render active course card containers populated with live data.

### Checkpoint 6 Verification Checklist

- ☐ The Network tab logs a successful **200 OK** HTTP request pointing to your .NET Web API
- ☐ Course cards render dynamically with real data (confirming your hardcoded mock array is disconnected)
- ☐ The loading spinner component appears briefly on screen before your data structures finish rendering
- ☐ Terminating your .NET API process and refreshing the browser causes the fallback error message block to render instead

